

Лабораторная работа №5

Разработка интерпретатора.

Теоретическая часть

Интерпретаторы важны по нескольким веским причинам. Во-первых, они могут обеспечивать удобную интерактивную среду (что зачастую удобнее, чем просто компилятор языка), Во-вторых, интерпретаторы языка обеспечивают превосходные интерактивные отладочные возможности. Интерпретатор позволяет, например, динамично устанавливать значения переменных и условий. В-третьих, большинство языков-запросов к базам данных работают в режиме интерпретации.

В данной работе будут даны общие методы и подходы к реализации интерпретатора на примере языка, который условно назовём CBAS (описание структуры программы на этом языке будет дано ниже). Язык представляет собой смесь языков BASIC и C, и упрощён почти до предела.

При разработке интерпретатора следует учесть тот факт, что каждая лексема имеет два формата: внешний и внутренний. Внешний формат – это строка символов, с помощью которой вы пишете программы на каком-либо языке программирования. Например, «while» – это внешняя форма ключевого слова while языка C. Можно построить интерпретатор из расчёта, что каждая лексема используется во внешнем формате, но это типичное решение проблемы программистом-непрофессионалом. Студентам четвёртого курса следует ориентироваться на внутренний формат лексемы, который является просто числом, и разрабатывать интерпретатор исходя из этой профессиональной точки зрения на проблему. Этот подход заключается в том, что каждой лексеме присваивается внутренний номер. Например, ключевое слово while может иметь порядковый номер 1, if – 2 и т.д. Преимущество внутреннего формата заключается в том, что программы, обрабатывающие числа, быстрее программ, обрабатывающих строки. Для реализации такого подхода необходима функция, которая берет из входного потока данных очередную лексему и преобразует ее из внешнего формата во внутренний.

Очень важно знать, какой тип лексемы возвращён. Например, анализатору выражений необходимо знать, является ли следующая лексема числом, оператором или переменной. Значение типа лексемы для процесса анализа в целом станет очевидным, когда вы приступите непосредственно к разработке интерпретатора.

Некоторые функции интерпретатора нуждаются в повторном просмотре лексемы. В этом случае лексема должна быть возвращена во входной поток. Например, это необходимо, когда очередная лексема является началом выражения. Тогда она помещается обратно во входной поток и вызывается функция обработки и вычисления этого выражения.

Спецификация языка CBAS

<character>: 'a' 'b' 'c' 'd' 'e' 'f' 'g' 'h' 'i' 'j' 'k' 'l' 'm' 'n' 'o' 'p' 'q' 'r' 's' 't' 'u' 'v' 'w' 'x' 'y' 'z' 'A' 'B' 'C' 'D' 'E' 'F' 'G' 'H' 'I' 'J' 'K' 'L' 'M' 'N' 'O' 'P' 'Q' 'R' 'S' 'T' 'U' 'V' 'W' 'X' 'Y' 'Z' '_'
<id_end>:

```

| <character><id_end>
<identifier>: <character><id_end>
<digit>: '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
<number_end>:
| <digit><number_end>
<number>: <digit><number_end>
<expression>: <term> '+' <expression>
| <term> '-' <expression>
| <term>
<term>: <factor> '*' <term>
| <factor> '/' <term>
| <factor>
<factor>: <identifier> | <number> | (<expression>)
<relop>: '<' | '>' | '=' | '!='
<bool_expression>: <expression> <relop> <expression>
<assign>: <identifier> '=' <expression>
<string>: '"' <любое количество отличных от '"' символов> '"'
<print>: 'print' <print_end> ';'
<print_end>: <string>
| <expression>
| <string> ',' <print_end>
| <expression> ',' <print_end>
<scan>: 'scan' <identifier> ';'
<for>: 'for' <identifier> '=' <expression> 'to' <expression>
<if>: 'if' <bool_expression>
<else>:
| 'else' '{' <statement> '}'
<statement>:
| <print> <statement>
| <scan> <statement>
| <assign> <statement>
| <for> '{' <statement> '}'
| <if> '{' <statement> '}' <else>
<program>: <statement>

```

Все переменные состоят только из букв и символа подчёркивания и являются глобальными. Такая конструкция, как объявление переменных, в языке CBAS не предусмотрена. Все переменные имеют тип данных, аналогичный `int`. Транслятор при транслировании исходной программы строит таблицу переменных. Таблица представляет собой пары «имя-значение». При обнаружении идентификатора транслятор просматривает таблицу в поисках такого же имени. Если его там нет, то создаётся новое вхождение в таблицу, и переменной присваивается какое-то начальное значение (например, 0).

Оператор **print** служит для вывода информации на экран. Он может печатать как символьные строки, так и значения выражений.

Оператор **scan** служит для считывания с клавиатуры значений в переменную. Выполнение программы приостанавливается, пока пользователь не введёт какое-либо число (т.к. все переменные имеют тип данных `int`).

Цикл **for** работает следующим образом. Переменной в цикле присваивается начальное значение, которое инкрементируется на единицу при каждой итерации. Выход из цикла происходит, когда переменная цикла БОЛЬШЕ ИЛИ РАВНА значению верхней границы цикла.

Некоторые особенности построения интерпретаторов.

Как уже упоминалось, интерпретатор оперирует внутренним представлением для лексем. Для преобразования во внутренний формат используется таблица

```
struct commands { /* Вспомогательная структура ключевых слов анализатора */
    std::string command;
    int token;
} table[];
```

Основной цикл работы интерпретатора состоит в пошаговом выполнении всех инструкций исходной программы. Все интерпретаторы выполняют операции путем считывания лексемы программы и выбора необходимой функции для ее выполнения. Например, если очередная лексема – это ключевое слово 'if', то запускается функция `process_if()`. В качестве параметра функция принимает указатель на обрабатываемую часть программы. Это может быть номер строки и символа в файле, какая-либо другая информация. Т.к. язык CBAS подразумевает вложенность конструкций, то каждая функция обработки той или иной лексемы вместе с выполнением специфических для данной лексемы функций должна вызывать общую функцию обработки. Пусть, например, программа состоит исключительно из циклов `for` и условий `if`. Тогда интерпретатор условно можно представить в виде следующих функций:

```
void process_if(char**);
void process_for(char**);

void main_loop(char* programm){
    int token = get_next_lexem(&program);
    switch (token){
        case IF:
            process_if(&program);
            break;
        case FOR:
            process_for(&program);
            break;
        default:
            //ошибка
    }
}

void process_if(char** pointer){
    //специфичные действия для if
    main_loop(*pointer);
}

void process_for(char** pointer){
```

```
//специфичные действия для for
main_loop(*pointer);
}
```

Циклы

В языке CBAS, точно также как и в C, допускается вложенность цикла for. Основной изюминкой при реализации этого оператора является сохранение информации о каждом вложенном цикле со ссылкой на внешний цикл. Для реализации этого используется стековая структура, которая работает следующим образом: начало цикла, информация о значении управляющей переменной цикла и ее конечном значении, а также место расположения цикла в теле программы заносятся в стек.

Каждый раз, при достижении '}', соответствующей концу цикла for, из стека извлекается информация о значении управляющей переменной, затем ее значение пересчитывается и сравнивается с конечным значением цикла. Если значение управляющей переменной цикла достигло своего конечного значения, выполнение цикла прекращается и выполняется оператор программы, следующий за циклом. В противном случае, в стек заносится новая информация, и выполнение цикла начинается с его начала. Таким же образом обеспечивается интерпретация и выполнение вложенных циклов. В стекоподобной структуре вложенных циклов каждый for должен быть закрыт соответствующей '}'.

Для реализации оператора цикла for стек должен иметь следующую структуру:

```
struct for_stack {
    int var; /* счетчик цикла */
    int target; /* конечное значение */
    char* location;
} fstack[FOR_COUNT]; /* стек для цикла for */
int ftop; /* индекс начала стека FOR */
```

Константа FOR_COUNT ограничивает уровень вложенности цикла. Переменная ftop всегда имеет значение индекса начала стека.

Вместо явного использования стековой структуры, можно воспользоваться механизмом локальных переменных языка, на котором реализуется интерпретатор. В этом случае вложенные операторы обрабатываются с помощью рекурсивного вызова соответствующих процедур, и необходимая информация сохраняется в локальных переменных вызывающей процедуры.

Порядок построения выражений

Имеется много вариантов анализа и вычисления выражений. Для использования полного синтаксического анализатора рекурсивного спуска мы должны представить выражение в виде рекурсивной структуры данных. Это означает, что выражение определяется в терминах самого себя. Если выражение можно определить с использованием только символов "+", "-", "*", "/" и скобок, то все выражения могут быть определены с использованием следующих правил:

Выражение => Терм [+Терм][-Терм]
Терм => Фактор [*Фактор[/Фактор]
Фактор => Переменная, Число или (Выражение)

Очевидно, что некоторые части в выражении могут отсутствовать вообще. Квадратные скобки означают именно такие необязательные элементы выражения. Символ '=>' имеет смысл "производит".

Фактически, выше перечислены правила, которые обычно называют правилами вывода выражения. В соответствии с этими правилами терм можно определить так: "Терм является произведением или отношением факторов".

Приоритет операторов безусловен в описанных выражениях, то есть вложенные элементы включают операторы с более высоким приоритетом.

В связи с этим рассмотрим ряд примеров. Выражение

$$10+5*B$$

содержит два термина: "10" и "5*B". Они, в свою очередь, состоят из трех факторов: "10", "5" и "B", содержащих два числа и одну переменную. В другом случае выражение

$$14*(7-C)$$

содержит два фактора "14" и "(7-C)", которые, в свою очередь, состоят из числа и выражения в скобках. Выражение в скобках вычисляется как разность числа и переменной.

Можно преобразовать правила вывода выражений в множество общих рекурсивных функций, что и является зачастую основной формой синтаксического анализатора рекурсивного спуска. На каждом шаге анализатор такого типа выполняет специфические операции в соответствии с установленными алгебраическими правилами. Работу этого процесса можно рассмотреть на примере анализа выражения и выполнения арифметических операций. Пусть на вход анализатора поступает следующее выражение:

$$9/3-(100+56)$$

Анализатор в этом случае будет работать по такой схеме:

1. Берем первый терм: "9/3".
2. Берем каждый фактор и выполняем деление чисел, получаем результат "3".
3. Берем второй терм: "(100+56)". В этой точке стартует рекурсивный анализ второго выражения.
4. Берем каждый фактор и суммируем их между собой, получаем результат 156
5. Берем число, вернувшееся из рекурсии, и вычитаем его из первого: 3-156. Получаем итоговый результат "-153".

Вы должны помнить две основные идеи рекурсивного разбора выражений:

1. приоритет операторов является безусловным в продукционных правилах и определен в них;
2. этот метод синтаксического анализа и вычисления выражений очень похож на тот, который вы сами используете для выполнения таких же операций.

Задание на лабораторную работу

Создать интерпретатор языка CBAS.

Входные данные: исходный текст программы на языке CBAS.

Выходные данные: консоль с результатами ввода-вывода исходной программы, либо сообщения об ошибках, присутствующих в программе.